

Engine with Fluid Hybrid Execution

Anonymous Submission

Abstract—The transparency required for public blockchain verification inherently conflicts with the confidentiality demanded by many transactions and smart-contract applications. Secure multi-party computation (MPC) and fully homomorphic encryption (FHE) provide promising approaches to reconciling this tension, yet existing confidential smart-contract systems often fall short of the decentralization expected from blockchains, as MPC execution and FHE key management are typically entrusted to a fixed subset of nodes. We present **PrivFlux**, a privacy-preserving smart-contract engine that combines hybrid FHE-MPC execution with rotating execution committees. **PrivFlux** supports confidential contract variables represented as either FHE ciphertexts or secret sharings, enabling contract operations to be executed over protected data without exposing their underlying values. To support decentralized committee rotation, we further design a dynamic state-handoff mechanism that securely transfers secret-shared values and distributed FHE key shares to an incoming committee without reconstructing the underlying secrets.

I. INTRODUCTION

Blockchains and smart contracts were originally designed around transparent replication and public verification rather than data confidentiality. As a result, transaction details, account activities, contract calls, and state updates are propagated among blockchain nodes and may reveal sensitive information about users and their interactions. This concern becomes more pronounced as blockchain applications and on-chain data continue to grow: an increasing volume of user-related data is transmitted, processed, and retained on publicly observable ledgers. Recently, privacy-preserving computation techniques—most notably secure multi-party computation (MPC) [?], [?], [?] and homomorphic encryption (HE) [?], [?—have therefore been increasingly adopted to protect the confidentiality of blockchain transactions and smart-contract execution [1]–[5].

Existing privacy-preserving blockchain systems follow several complementary directions. Proof-based designs hide transaction or contract witnesses while retaining public verification, and trusted-hardware designs execute private logic inside an isolated environment [6]–[9]. MPC-based systems distribute private execution among multiple parties, whereas HE/FHE-based systems keep contract inputs and state encrypted during

computation [1]–[5], [10], [11]. Despite this progress, privacy-preserving contracts continue to face two practical challenges. First, many designs delegate private execution or key management to a manager, a fixed committee, a coprocessor, or a dedicated KMS, creating a centralized or long-lived trust boundary. Second, privacy protection introduces substantial performance costs: MPC requires interaction and communication, whereas FHE is efficient for some arithmetic workloads but expensive for many non-arithmetic operators. Consequently, a privacy-preserving contract system must improve both decentralization and execution efficiency rather than address confidentiality alone.

Therefore, we propose PrivFlux, which is a confidential smart-contract runtime driven by a public, contract-defined operator sequence. The host chain stores the contract and public data references. Private runtime data remain off chain in one of two representations: FHE ciphertexts or secret shares. The active committee resolves the references and executes each FHE or MPC operator. An explicit FHE-ciphertext-to-sharing (H2S) or sharing-to-FHE-ciphertext (S2H) step is used only when the next operator requires the other representation. Unlike schemes such as FHEVM, PrivFlux does not require a KMS. Instead, FHE secret keys are treated as secret-shared (SS) data, while FHE ciphertexts are treated as public data. To reveal an FHE-encrypted value to a user, the ciphertext is first converted into an SS representation using the secret-shared FHE key and then opened to the user.

To remove the reliance on a fixed, long-lived execution committee, PrivFlux divides execution into epochs and rotates the parties responsible for private computation. At the beginning of epoch ℓ , cryptographic sortition selects an active committee S^ℓ from the candidate computing nodes. This committee evaluates all private-contract tasks submitted during the epoch and stores the private values produced by those tasks. Consequently, private execution and data custody are distributed across different committees over time rather than assigned permanently to one group. If the active committee of a later epoch needs a private value stored by an earlier committee, the runtime invokes the handoff protocol. Handoff transfers the value from its current holder committee to the new active committee without reconstructing the underlying plaintext, after which the new committee can continue the contract execution. This epoch-based selection and state-transfer mechanism removes the need for a fixed executor while preserving access to private state across committee changes. Overall, the contributions of PrivFlux are summarized as follows:

- We design and implement an end-to-end solution for privacy-preserving smart contracts. The solution covers the complete contract lifecycle, including contract deployment, private-value registration, contract invocation, off-chain execution, storage, and result retrieval.
- We design a secret-sharing-based handoff mechanism to realize a decentralized privacy-preserving smart-contract engine. Cryptographic sortition selects a new active committee for each epoch, and the handoff protocol transfers protected state from its current holder committee to the incoming committee without reconstructing the underlying plaintext. This mechanism allows contract execution and private-state custody to rotate over time instead of relying on a fixed execution committee or centralized key-management service.
- We support hybrid MPC-FHE execution and build a complete system that integrates this hybrid model.

a) Organization: The remainder of this section reviews related privacy-preserving blockchain systems, introduces the cryptographic preliminaries, and states the threat model. Section II presents the architecture of PrivFlux, its on-chain and off-chain data structures, a confidential-token transfer example, and the end-to-end contract workflow. Section III describes the runtime protocols, including committee handoff, shared offline preprocessing, hybrid FHE-MPC execution, and representation conversion. Section IV presents the implementation and evaluates the protocol-level and end-to-end performance of the system.

A. Preliminaries

a) Notation: We write vectors in bold and use $x \stackrel{\$}{\leftarrow} \mathcal{D}$ for sampling x from a distribution $\mathcal{D}$. Let λ be the security parameter and let $\mathcal{P} = \{P_1, \dots, P_n\}$ be the participating parties, of which at most t may be corrupted. The MPC protocols perform arithmetic in a prime field $\mathbb{F}_q$. The notation $[x]^{d,q,\mathcal{P}}$ denotes a degree- d Shamir sharing held by $\mathcal{P}$ over $\mathbb{F}_q$. When clear from the context, we omit the superscripts d, q or $\mathcal{P}$.

b) t -threshold secret sharing: Secret sharing lets a group hold a secret without giving the complete value to any single party. A t -threshold scheme protects the secret against any group of at most t parties. Any $t + 1$ parties can recover it, while the joint view of any set of at most t parties is independent of the secret.

Shamir secret sharing. Let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q \setminus \{0\}$ be distinct public evaluation points. To share $x \in \mathbb{F}_q$ with threshold t , sample $f_x(X) = x + a_1X + \dots + a_tX^t$, where $a_1, \dots, a_t \stackrel{\$}{\leftarrow} \mathbb{F}_q$, and give party P_i the share $[x]_i^{t,q} = f_x(\alpha_i)$. Any $t+1$ valid shares reconstruct $f_x(0) = x$ by Lagrange interpolation. Any set of at most t shares is statistically independent of x [12].

Replicated secret sharing. Replicated sharing gives several parties a common additive component. Let $\mathcal{H}_t = \{A \subseteq [n] : |A| = n - t\}$. A replicated sharing of $x \in \mathbb{F}_q$ consists of components $\{x_A\}_{A \in \mathcal{H}_t}$ sampled uniformly subject to

$$x = \sum_{A \in \mathcal{H}_t} x_A, \quad (1)$$

where party P_i holds x_A exactly when $i \in A$. Every set of $t+1$ parties collectively holds every component and can reconstruct the sum. A coalition of at most t parties misses at least one independently randomized component. This representation is used later to derive non-interactive random Shamir sharings and correlated preprocessing material.

Linearity. For the operations used by the Shamir and replicated-sharing instantiations, parties can locally derive shares of $ax + by$ from shares of x and y for public coefficients a and b .

c) Authentication: Authentication lets the parties detect inconsistent shares and an incorrect opening. After values enter the arithmetic computation, PrivFlux uses an information-theoretic MAC over $\mathbb{F}_q$. Let $\alpha \in \mathbb{F}_q$ be a global secret MAC key that is itself secret-shared. An authenticated sharing of $x \in \mathbb{F}_q$ is

$$\langle\langle x \rangle\rangle = ([x]^q, [\gamma_x]^q), \quad \gamma_x = \alpha x. \quad (2)$$

Linearity gives authenticated sharings of public linear combinations without interaction. After reconstructing an intended opening x , the parties open $[\gamma_x]^q - x[\alpha]^q$ and accept only if the result is zero. This authentication method is used by active-secure arithmetic MPC, including SPDZ-style protocols [13], [14].

d) FHE: PrivFlux uses a ring-based threshold-FHE scheme. The MPC/Shamir modulus q above is distinct from the FHE ciphertext modulus Q . Define $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N + 1)$. A message $m \in \mathbb{F}_q$ is embedded into $\mathcal{R}_Q$ by Encode and encrypted under the public key as a two-component ciphertext

$$\text{ct} = \text{Enc}_{\text{pk}}(m) = (a(X), b(X)) \in \mathcal{R}_Q^2. \quad (3)$$

The public evaluation key allows the runtime to evaluate the contract's FHE operators directly on ciphertexts. Under the sign convention used throughout this paper, decryption first computes

$$\begin{aligned} u(X) &= b(X) - a(X)s(X) \\ &= \Delta \text{Encode}(m) + e(X) \pmod{Q}, \end{aligned} \quad (4)$$

where the FHE secret key $s(X) \in \mathcal{R}_Q$ and noise $e(X) \in \mathcal{R}_Q$ are polynomials, and $\Delta = \lfloor Q/q \rfloor$ is the encoding scale. Applying Decode coefficient-wise or slot-wise recovers m when the corresponding noise is within the correctness bound. The active committee holds a coefficient-wise threshold sharing of the ring element, denoted $[s(X)]^{t,\mathcal{R}_Q,\mathcal{S}}$; for an RNS ciphertext, the sharing is maintained independently modulo each coefficient prime. Thus party P_i holds only $[s(X)]_i^{t,\mathcal{R}_Q,\mathcal{S}}$ and computes its local ring share $[u(X)]_i = b(X) - a(X)[s(X)]_i$. This shared decryption intermediate is the only FHE-specific relation needed by the later FHE-to-secret-sharing conversion.

e) Fluid security and dynamic committees: With a static committee, the same parties retain their shares throughout the execution. In a dynamic system, an adversary could compromise different parties in different epochs and accumulate obsolete shares. A fluid handoff prevents obsolete shares from being accumulated when the active committee changes only

together with secure erasure. Once a handoff is finalized, honest outgoing parties erase their old value shares, MAC shares, FHE secret-key shares, one-time masks, temporary protocol state, and unused epoch-specific preprocessing. Consequently, a mobile adversary cannot combine shares obtained from different epochs into one qualified reconstruction set.

f) Preprocessing: The protocols use an offline preprocessing phase. Before private inputs are available, the parties prepare data-independent random sharings and correlated randomness required by the online phase. All of our protocols are designed for the handoff setting between two committees, which generate the required correlated randomness during the offline phase.

B. Threat Model

At each execution epoch, a new committee is selected via cryptographic sortition. We assume that, for every selected committee of n parties, at most $t < n/2$ parties are corrupted; the set of corrupted parties may vary across epochs, as long as this bound holds for each committee. Our basic protocols guarantee privacy against semi-honest adversaries that follow the prescribed protocol while attempting to infer additional information from their local views. They further preserve confidentiality in the presence of actively deviating parties, but without authentication such deviations may compromise correctness. To achieve security against malicious adversaries, we augment the protocols with authentication mechanisms that allow honest parties to detect inconsistent shares or invalid computation results.

a) Secure-erasure assumption: We assume that an honest runtime party can irreversibly erase designated local state. After the incoming committee validates a handoff and issues its completion certificate, every honest outgoing party securely erases all obsolete shares of the transferred values and FHE secret-key polynomial, associated MAC shares, consumed correlated randomness, noise-flooding samples, temporary masked values, and unused preprocessing bound to the old epoch. Epoch-specific PRSS state is either erased or advanced with a forward-secure update so that later corruption cannot recover earlier randomness. Long-term public identities, signing keys, public transcripts, and audit records are not part of this erasure. This assumption is required for security against a mobile adversary whose corrupted set changes between epochs: without erasure, such an adversary could retain old shares and eventually collect more than t shares of the same pre-handoff state. The protocol therefore provides proactive privacy only while the per-epoch corruption bound and secure-erasure assumption both hold.

II. SYSTEM OVERVIEW

In this work, we propose PrivFlux, which is a confidential smart-contract runtime that allows users to deploy and execute privacy contracts with private inputs. As shown in Fig. 1, the system comprises a user interface, a host chain, a runtime and off-chain storage. The host chain provides public coordination by recording contracts, data references, invocations, and result

identifiers, while confidential payloads remain off chain as FHE ciphertexts or secret-shared (SS) records. The runtime resolves the public references and dispatches each contract-defined operator to the active committee, which evaluates FHE and MPC operators, performs explicit conversions between the two representations when required.

Compared with typical privacy-preserving smart-contract systems, our framework has three distinctive features:

- All encrypted data are stored off-chain, while the blockchain only maintains references to the corresponding off-chain data, thereby avoiding the overhead of storing and processing ciphertexts directly on-chain.
- We adopt a hybrid FHE-MPC architecture for privacy-preserving smart-contract execution, allowing different cryptographic techniques to be used according to the characteristics of individual computations.
- We introduce a handoff mechanism that securely transfers cryptographic states, including secret shares and distributed FHE key material, across dynamically selected execution committees. This eliminates the reliance on centralized key management and fixed contract-execution committees.

Our implemented system fully supports the following workflow: i) deploying a privacy-preserving smart contract, ii) invoking the contract with private inputs to perform operations such as encrypted token transfers, and iii) querying the contract execution results, including functionalities such as encrypted token balance queries.

A. System Architecture and Components

The components in Fig. 1 jointly provide the system interface. The host chain provides public coordination, whereas the active committee performs cryptographic operations on off-chain values. Their complete interaction order is specified in Section II-D.

a) User interface: The user supplies a privacy-preserving contract, private inputs, or a variable query request. Through the interface, the user deploys a contract to host chain, the private variable on the host chain, while sending the corresponding private data to the off-chain storage in the form of either FHE ciphertexts or secret shares, invokes a callable function over private data, and requests an authorized opening.

b) Host chain: The host chain is the public coordination layer. The Privacy Contract records the callable program, variable-to-identifier bindings, and the SS, FHE, and conversion operators. The Address Registry maps each private data from its public reference to offchain address. Contract deployment returns a public contract address, while data registration returns a public data reference.

c) Runtime: The runtime consists of all off-chain computing nodes, each associated with an on-chain address. These nodes monitor on-chain events, such as users uploading encrypted data or invoking privacy-preserving smart contracts, and perform the corresponding cryptographic computations. Not all runtime nodes participate in each computation. Instead, at any given time, a subset of runtime nodes is selected to

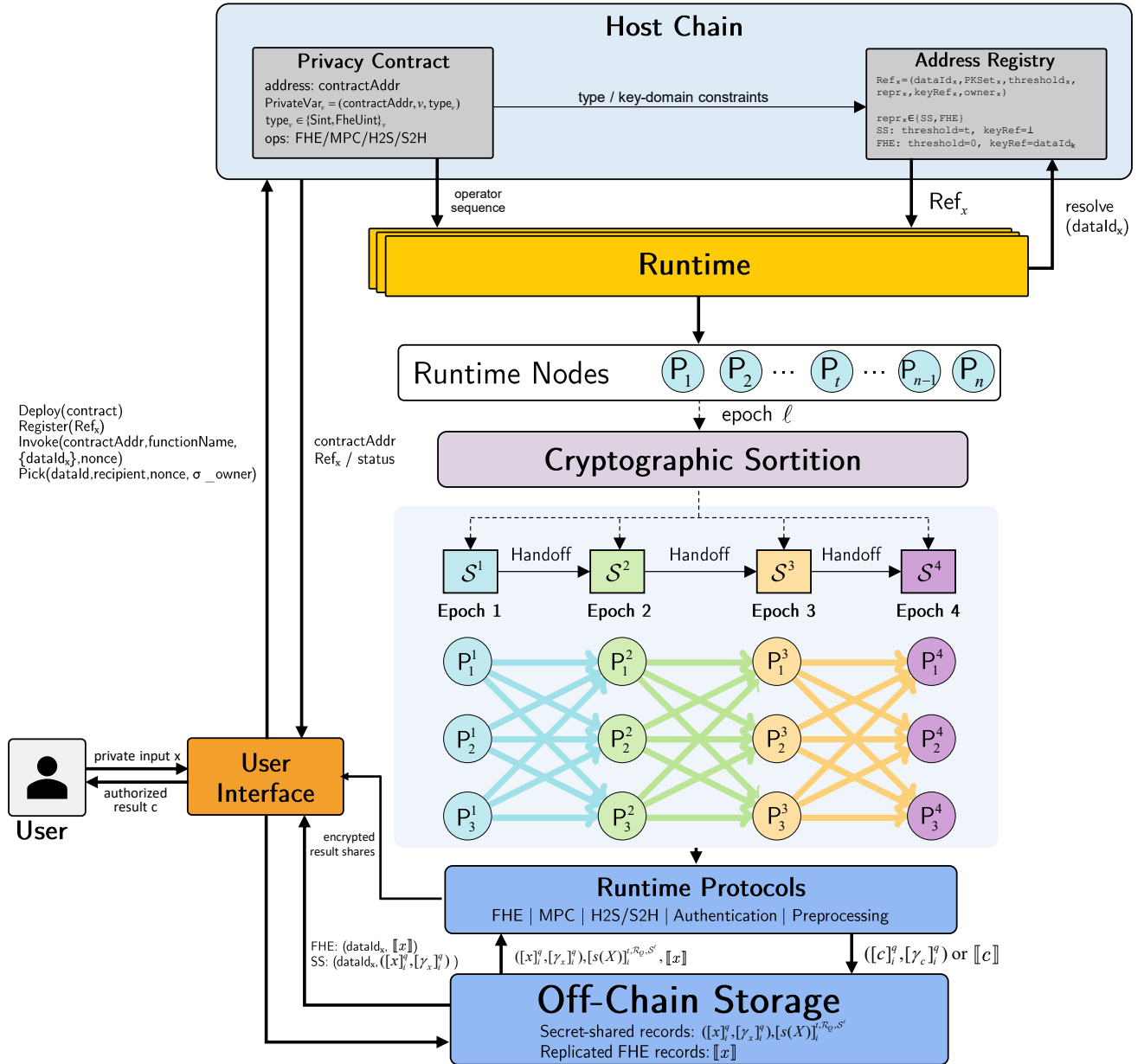


Fig. 1. System components and dataflow. Arrow labels identify the sender, receiver, and transmitted public references or protected values. The host chain stores public contract state and data references; party P_i stores only its own secret shares.

form an active execution committee responsible for carrying out the secure computation. The active lifetime of an execution committee is referred to as an *epoch*. At the end of each epoch, the runtime selects a new execution committee from the set of runtime nodes.

The runtime supports both FHE and secret-sharing (SS) evaluation. FHE ciphertexts (as well as the corresponding public keys) appearing in a contract are treated as public values, whereas the associated FHE secret-key polynomial $s(X) \in \mathcal{R}_Q$ is stored off chain as a coefficient-wise threshold sharing. Conversions between FHE ciphertexts and secret-shared values are explicitly specified at the contract level, corresponding to H2S or S2H operators. The runtime follows

this public sequence for that function. The active committee evaluates FHE operators on ciphertexts and MPC operators on shares. Threshold-FHE key shares use the same SS data-reference format as other secret-shared values. The Privacy Contract associates the key data identifier with its public and evaluation keys and binds each FHE operator to the corresponding key record. Each FHE ciphertext record names that SS key-share record in its keyRef field. The active committee holds only local secret-key shares. These shares are used only inside committee protocols, including H2S, and are never released to a user by the Pick interface.

Cryptographic sortition: The active computing committee for each epoch is selected via cryptographic sortition. At

the start of epoch ℓ , cryptographic sortition selects $\mathcal{S}^\ell = \{P_1, \dots, P_n\}$ and publishes it.

Authentication: We implement correctness verification for both the computation results produced by the active committee and the outputs of the handoff procedure. More precisely, data handed off from $\mathcal{S}_1$ to $\mathcal{S}_2$ are verified during the subsequent handoff. Please refer to Section III for details.

d) Off-chain storage: The off-chain storage nodes are physically co-located with the runtime execution nodes. Each on-chain variable is associated with a corresponding value stored off-chain. Values generated by the runtime during an epoch are stored by the active execution committee for that epoch. When a subsequent epoch needs to access a previously generated variable, a handoff is performed to securely transfer the corresponding data from the committee that currently stores it to the newly active execution committee. The off-chain data are initially provided by users. After registering a variable on the host chain, the user sends the corresponding data to the designated off-chain storage nodes in the form of either an FHE ciphertext or secret shares.

e) Confidential-token contract: The running contract maintains a confidential-token balance for each account. Creating an account initializes its encrypted balance to zero. The contract supports private transfers: it checks whether the sender's encrypted balance is sufficient, subtracts the encrypted amount from the sender, and adds the same amount to the recipient. The balance and amount appearing in the contract are typed private variables used to express this logic; the concrete ciphertexts are separate private values selected by their data identifiers at invocation.

Figure 2 gives a schematic excerpt of the privacy-preserving Swap contract. The excerpt exposes only the contract-level interface and protected operators; the runtime resolves these operators to the FHE, MPC, and conversion protocols described later in the paper.

B. Data Structures

Below, we present the concrete data structures used in PrivFlux, including deployed privacy-preserving smart contracts, private variables, and user-initiated contract invocation requests.

a) Privacy contract: A deployed Privacy Contract has a public contract address. It contains callable functions, private-variable declarations, and public operator sequences. Each function fixes the order of its FHE and MPC operators, its explicit H2S or S2H boundaries, and the FHE key reference used by each FHE operator. The contract specifies how private values are processed without storing the values. Private variables serve as typed symbolic operands in these operator sequences; the concrete protected values are supplied separately when a function is invoked.

b) Data reference: For each private value x , the Address Registry stores the public reference

$$\text{Ref}_x = (\text{dataId}_x, \text{PKSet}_x, \text{threshold}_x, \text{repr}_x, \text{keyRef}_x, \text{owner}_x). \quad (5)$$

```

ConfidentialSwap
1  privacy contract ConfidentialSwap {
2    private FheUint balance[address];
3
4    function createAccount() public {
5      balance[msg.sender] = FHE.encrypt(0);
6    }
7
8    function deposit(FheUint amount) public {
9      receiveEncryptedToken(msg.sender, amount);
10     balance[msg.sender] =
11       FHE.add(balance[msg.sender], amount);
12   }
13
14   function withdraw(FheUint amount) public {
15     SecretBool sufficient =
16       H2S(FHE.ge(balance[msg.sender], amount));
17     require(sufficient);
18     balance[msg.sender] =
19       FHE.sub(balance[msg.sender], amount);
20     sendEncryptedToken(msg.sender, amount);
21   }
22
23   function getBalance() public {
24     Sint balanceShare = H2S(balance[msg.sender]);
25     return Pick(balanceShare);
26   }
27 }

```

Fig. 2. Swap contract for encrypted-token account creation, deposit, withdrawal, and balance retrieval.

Here $\text{repr}_x \in \{\text{SS}, \text{FHE}\}$. For an SS value, PKSet_x identifies the committee members and $\text{threshold}_x = t$ gives the Shamir threshold and degree. Any set of at most t shares hides the value, while at least $t + 1$ shares are needed to open it. An ordinary MPC value is stored as an authenticated sharing $\langle\langle x \rangle\rangle_q = ([x]^q, [\gamma_x]^q)$; its value shares, MAC shares, and the committee's MAC-key shares remain off chain. Its keyRef_x field is $\perp$. An FHE secret key is also registered with $\text{repr} = \text{SS}$ under its own dataId_k , but its payload is the coefficient-wise sharing $[s(X)]^{t, \mathcal{R}_Q, \mathcal{S}}$ of a ring element, not a scalar sharing over $\mathbb{F}_q$. For a replicated FHE ciphertext, PKSet_x identifies the storage or computation nodes and $\text{threshold}_x = 0$ marks an FHE record. Its keyRef_x names the corresponding SS key-share record. The owner field identifies the principal whose signature authorizes a Pick request. For an FHE reference, keyRef_x locates the secret-shared FHE key needed by H2S; it does not authorize release of that key. The registry accepts the reference only when the ciphertext and key records belong to the same FHE key domain declared by the Privacy Contract. When a new private value is created, the system obtains the active committee from the cryptographic-sortition result for the current epoch. If sortition selects $\mathcal{S}^\ell = \{P_1, \dots, P_n\}$, the variable's data reference is initialized with $\text{PKSet}_x = \{\text{PK}_1, \dots, \text{PK}_n\}$, where PK_i is the public key of party P_i . This field identifies the nodes to which the variable's off-chain ciphertext or shares are distributed and the committee responsible for its subsequent private computation.

c) Private Variable: A private variable and a private value are distinct objects. A private variable is a public, typed symbol used only to construct the contract's invocation logic: it names an input, intermediate result, or output position in the public operator sequence, but does not itself contain

protected data. For a variable named v , the host chain stores the declaration

$$\text{PrivateVar}_v = (\text{contractAddr}, v, \text{type}_v). \quad (6)$$

Here, contractAddr identifies the contract that defines the variable, v is the variable's name or position within that contract, and type_v is its logical contract type. For example, FheUint denotes an unsigned-integer variable evaluated in the FHE domain, whereas Sint denotes an integer variable evaluated in the secret-sharing domain. Each logical type determines its required confidential representation through $\text{repr}(\text{FheUint}) = \text{FHE}$ and $\text{repr}(\text{Sint}) = \text{SS}$. During an invocation, the caller supplies a dataId for each input variable. The runtime constructs a call-local mapping $v \mapsto \text{dataId}_x$, resolves Ref_x through the Address Registry, and verifies that $\text{repr}_x = \text{repr}(\text{type}_v)$ before substituting the value into the contract's operator sequence.

This mapping does not turn the variable into a value and does not copy the underlying protected payload onto the host chain. In particular, dataId , PKSet , threshold, key reference, owner, ciphertext, and secret shares are properties of the private value and its data reference, not of the private variable. Once the invocation is instantiated, the variable only determines how the referenced values flow through the contract logic.

d) Invocation record: After the inputs are uploaded, a user invokes a callable contract function with

$$\text{Invoke}(\text{contractAddr}, \text{functionName}, \{\text{dataId}_x\}, \text{nonce}). \quad (7)$$

Here, contractAddr identifies the deployed Privacy Contract whose state and access policy govern the call. The functionName field selects one callable entry point and, consequently, the public sequence of FHE, MPC, and conversion operators declared for that function. The collection $\{\text{dataId}_x\}$ contains the public identifiers of the private input objects supplied to the function. Each identifier is matched to a formal input variable, resolved through the Address Registry, and checked against the representation declared for that private variable. Finally, nonce is a fresh value supplied for this call; it distinguishes otherwise identical invocations and allows duplicate or replayed requests to be rejected. The off-chain runtime detects the corresponding function invocation event, after which the active computing committee executes the computation off-chain and produces the result.

e) Pick request: To retrieve a private result, a user submits

$$\text{Pick}(\text{dataId}, \text{recipient}, \text{nonce}, \sigma_{\text{owner}}). \quad (8)$$

The user is authenticated as the transaction sender. The owner's signature covers the request tag, dataId , user identity, recipient, and nonce. The parties named by PKSet read owner from the data reference, verify the owner signature, and reject a reused nonce. For an SS result, they return a qualified set of opening shares encrypted for the recipient only after the authenticated opening check succeeds. For an FHE result,

the parties use keyRef to resolve the committee-held FHE-key sharing and invoke H2S on the requested ciphertext. This produces a temporary SS result under the active committee. The parties authenticate that result and return its opening shares, encrypted for the named recipient, exactly as in the SS case. The recipient reconstructs only the requested plaintext; Pick returns no FHE secret-key share.

C. Running Example

Consider the confidential transfer in Fig. 3. Alice transfers a private amount m to Bob. Their current encrypted balances are the private values b_A and b_B , referenced by dataId_{b_A} and dataId_{b_B} ; the encrypted transfer amount is referenced by dataId_m . These identifiers name values, not contract variables. The contract separately declares three logical operands:

$$\begin{aligned} \text{PrivateVar}_v &= (\text{contractAddr}, v, \text{FheUint}), \\ v &\in \{\text{bal}_A, \text{bal}_B, \text{amount}\}. \end{aligned} \quad (9)$$

All three variables have logical type FheUint and therefore require FHE data references. The Address Registry stores one Ref for each ciphertext. Each reference records its data identifier, the current committee's PKSet , $\text{repr} = \text{FHE}$, and the key reference for the contract's FHE key domain.

Alice invokes transfer with

$$\text{Invoke}(\text{contractAddr}, \text{transfer}, \{\text{dataId}_{b_A}, \text{dataId}_{b_B}, \text{dataId}_m\}, \text{nonce}). \quad (10)$$

For this call, the runtime constructs the temporary mappings $\text{bal}_A \mapsto \text{dataId}_{b_A}$, $\text{bal}_B \mapsto \text{dataId}_{b_B}$, and $\text{amount} \mapsto \text{dataId}_m$. It resolves the three data references and rejects the call unless their representations and key domains match the variable declarations and the contract policy.

The active committee then follows the operator sequence in the contract. It first evaluates the protected predicate $b_A \geq m$ and converts the encrypted predicate to the secret-shared form required by the private check. If the check fails, neither balance is changed. Otherwise, the committee evaluates

$$b_A \leftarrow b_A - m, \quad b_B \leftarrow b_B + m, \quad (11)$$

and writes the resulting ciphertexts back under the original balance data identifiers. Thus, the contract variables determine the transfer logic, while dataId_{b_A} and dataId_{b_B} continue to identify the actual encrypted balances before and after the update. The amount value is read-only and remains unchanged.

D. End-to-End Dataflow

The following lifecycle assembles the functions and component responsibilities above into one complete execution sequence.

a) Contract deployment: The user sends the contract program and its callable functions to the host chain, which returns contractAddr . The deployed program fixes the variable declarations and operator sequence. Deployment creates no input data identifiers.

```

ConfidentialToken
1  privacy contract ConfidentialToken {
2    private FheUint balance[address];
3
4
5    function transfer(address to,
6                      FheUint amount) public {
7      SecretBool sufficient =
8        H2S(FHE.ge(balance[msg.sender], amount));
9      require(sufficient);
10     balance[msg.sender] =
11       FHE.sub(balance[msg.sender], amount);
12     balance[to] = FHE.add(balance[to], amount);
13   }
14
15   function getBalance() public {
16     Sint balanceShare = H2S(balance[msg.sender]);
17     return Pick(balanceShare);
18   }
19
20 }

```

Fig. 3. Confidential-token transfer contract.

b) *Private-value registration*: Before invoking a contract, a user registers each concrete private value that will be supplied to a private variable. The SDK assigns the value a fresh dataId and determines its required representation from the corresponding variable type. It then reads the current sortition result S^ℓ and fills PKSet with the public keys of the active committee. For an FHE value, the SDK encrypts the value under the contract's FHE key domain, uploads the ciphertext to the designated off-chain nodes, and records the associated keyRef . For an SS value, it sends each committee member only that member's share and records the applicable threshold. In both cases, the confidential payload remains off chain.

The Address Registry accepts Ref_x only after the designated nodes acknowledge storage of the payload under dataId_x . Registration does not create or modify a private variable: it creates a value reference that can later be mapped to a compatible variable by an invocation. In the transfer example, this procedure registers Alice's balance b_A , Bob's balance b_B , and the amount m under dataId_{b_A} , dataId_{b_B} , and dataId_m , respectively.

c) *Invocation and output registration*: After upload, the user sends the host chain

$$\text{Invoke}(\text{contractAddr}, \text{transfer}, \{ \text{dataId}_{b_A}, \text{dataId}_{b_B}, \text{dataId}_m \}, \text{nonce}). \quad (12)$$

The host chain posts the invocation record. The runtime resolves the three data references and maps them to bal_A , bal_B , and amount for this call. After checking their types, representations, and key domains, it dispatches the transfer sequence to S^ℓ . The committee checks the protected balance condition and, if it succeeds, writes the updated ciphertexts back under dataId_{b_A} and dataId_{b_B} . No new identifier is created for either balance. The completion record reports whether the transfer completed or failed and identifies the balance records affected by the call.

d) *Pick and authorized opening*: For an SS result c , the user submits the owner-authorized Pick request in Eq. (8)

with $\text{dataId} = \text{dataId}_c$. At least $t + 1$ parties named by PKSet_c participate in the authenticated opening check. Only if it succeeds do they return encrypted result shares to the named recipient, who reconstructs the result. For an FHE result, the request names its ciphertext identifier. The runtime resolves the associated FHE-key sharing through keyRef and invokes H2S , producing a temporary SS representation of that ciphertext's plaintext under the active committee. The committee then applies the same authenticated-opening path and returns only the temporary value shares encrypted for the recipient. An invalid owner signature, a reused nonce, a mismatched key domain, or a failed authentication check causes the request to be rejected. The FHE secret key and all of its shares remain inside the committee throughout this procedure, so opening one balance does not enable the recipient to decrypt another user's ciphertext in the same key domain.

e) *Epoch transition*: Before handoff, sortition determines the incoming committee $S^{\ell+1}$. For a field operation f at the boundary, the offline phase prepares a cross-committee $\text{Prep}_{\text{HO}}^\ell$ pair. The outgoing committee evaluates f , masks its degree- d result with the source-side share of the pair, and sends a qualified set of masked shares to a designated dealer. The dealer reconstructs and broadcasts only $\delta = f(x) - r$. Each incoming party adds δ to its degree- t share of the same r . Choosing $f(x) = x$ gives ordinary state handoff. A coefficient-wise ring-domain instance of the same resharing pattern transfers the threshold sharing of the FHE secret-key polynomial because that key is registered as an SS record; it does not identify the ring key with an $\mathbb{F}_q$ scalar, reconstruct the key, or change the logical key identifier or public key. Ordinary authenticated MPC state additionally carries its MAC-tag sharing through the corresponding authenticated handoff. The incoming committee then prepares fresh preprocessing material for the new epoch. After successful handoff, the registry replaces PKSet in the live SS references, including the FHE-key SS record, with $S^{\ell+1}$. Data identifiers, public keys, owners, release policies, and replicated ciphertexts remain unchanged. The incoming committee then continues execution. Finally, after the incoming committee validates and durably installs the transferred state, it publishes a handoff-completion certificate. Honest members of the outgoing committee then securely erase old value, MAC, and FHE-key shares; temporary masked values; consumed masks and noise-flooding material; and unused preprocessing associated with the old epoch. Public transcripts and registry records remain available for audit. The system's protection against parties corrupted in different epochs relies on this secure-erasure step.

III. RUNTIME PROTOCOLS

This section presents the protocol layer that realizes the platform workflow described in Section II. These protocols allow PrivFlux to execute privacy-preserving contracts over off-chain values while the active committee changes across epochs. Together, they provide three capabilities required by the platform: confidential computation over FHE ciphertexts

Protocol 1 $[x]^{S_2} \leftarrow \Pi_{\text{Handoff}}([x]^{S_1})$

Input: Secret share $[x]^{S_1}$ held by S_1 .

Offline material: $([r]^{S_1}, [r]^{S_2}) \leftarrow \mathcal{F}_{\text{offline}}$

Output: Secret share $[x]^{S_2}$ held by S_2 .

Execution:

- Each party $P_i^{(1)}$ in S_1 calculates $[\delta]_i = [x]_i^{S_1} - [r]_i^{S_1}$ and sends $[\delta]_i$ to Dealer P_0 ;
- Dealer P_0 reconstruct δ by Lagrange interpolation and broadcast δ to S_2 ;
- After receiving δ , each $P_i^{(2)}$ in S_2 sets $[x]_i^{S_2} = [r]_i^{S_2} + \delta$;
- After S_2 validates and durably installs the new sharing, it issues a handoff-completion certificate. Each honest party in S_1 then securely erases its old shares and all consumed or unused epoch-specific handoff material.

Fig. 4. Secure Share Handoff Protocol

and secret-shared values, secure transfer of private state between committees, and conversion between the two representations when required by a contract.

The Basic Handoff Protocol transfers a secret-shared value from its current holder committee to the committee selected for the next epoch without opening the value. Its extensions combine state transfer with multiplication, FHE-secret-sharing conversion, and authentication, allowing a contract task to continue when either its committee or data representation changes. The shared offline-preprocessing protocols generate the one-time correlated randomness consumed by these online operations. In the complete platform, the runtime selects the appropriate protocol from the contract's operator sequence and the data references, while the active committee performs the cryptographic work and returns the private value needed by subsequent contract calls.

The remainder of this section first presents the Basic Handoff Protocol and its computation, conversion, and authentication cases, and then explains how their shared preprocessing material is generated.

A. Basic Handoff Protocol

We first introduce a basic handoff protocol based on Shamir secret sharing, whose goal is to transfer a secret-shared value from one committee to another. A straightforward solution in which the old committee directly sends its shares to the new committee is insecure. In particular, we assume that both the old and the new committees may contain up to t corrupted parties. If shares are directly transferred across committees, the adversary may accumulate more than t shares of the same sharing across the two committees and thereby reconstruct the underlying plaintext. To avoid this leakage, the transfer must instead be performed through a resharing procedure, such that the shares before and after resharing are independently

Protocol 2 $[z]^{t,S_2} \leftarrow \Pi_{\text{mult}}([x]^{t,S_1}, [y]^{t,S_1})$

Input: Secret share $[x]^{S_1}$ and $[y]^{S_1}$ held by S_1 .

Offline material: $([r]^{2t,S_1}, [r]^{t,S_2}) \leftarrow \mathcal{F}_{\text{offline}}$

Output: Secret share $[z]^{t,S_2}$ held by S_2 .

Execution:

- Each party $P_i^{(1)}$ in S_1 calculates $[\delta]_i = [x]_i^{t,q,S_1} \cdot [y]_i^{t,q,S_1} - [r]_i^{2t,q,S_1} \pmod{q}$ and sends $[\delta]_i$ to Dealer P_0 ;
- Dealer P_0 reconstruct δ by Lagrange interpolation and broadcast δ to S_2 ;
- After receiving δ , each $P_i^{(2)}$ in S_2 sets $[x]_i^{S_2} = [r]_i^{t,S_2} + \delta$;

Fig. 5. Secure Multiplication with Handoff Protocol

randomized representations of the same secret. To realize this functionality, we adapt the DN protocol [] to the handoff setting. Fig. 4 depicts this procedure. Let committees S_1 and S_2 hold correlated Shamir sharings $([r]^{S_1}, [r]^{S_2})$ of the same random value r . Given a secret sharing $[x]^{S_1}$ held by S_1 , the parties in S_1 locally compute $[x]^{S_1} - [r]^{S_1}$ and open the result to a designated party P_0 , who reconstructs $\delta = x - r$ and broadcasts it to S_2 . The parties in S_2 then locally compute

$$[x]^{S_2} = [r]^{S_2} + \delta,$$

thereby obtaining a fresh sharing of x under S_2 without revealing x . This completes the handoff of a secret-shared value from one committee to another. In the next subsection, we systematically discuss how to efficiently generate the required correlated randomness $([r]^{S_1}, [r]^{S_2})$ in the offline phase. We further observe that this handoff paradigm is not limited to transferring secret keys; rather, it provides a general mechanism that can be incorporated into a broad range of computations.

Handoff finalization includes secure erasure. The incoming committee first validates the transcript, installs the new shares, and publishes a completion certificate bound to the epoch and data identifiers. Only then do honest outgoing parties erase the old sharing, source-side masks, local reconstruction messages, and remaining preprocessing for that epoch. Public transcripts are retained for audit. This ordering prevents data loss if transfer fails while ensuring that a later corruption of an outgoing party does not recover an obsolete share. All multiplication, conversion, authenticated, and FHE-key handoffs use the same finalization rule.

The reason is that Shamir secret sharing supports the local evaluation of up to $k = \lfloor (n-1)/t \rfloor$ multiplications, after which the degree of the resulting sharing increases to $t \cdot k$. A resharing operation is then required to reduce the sharing degree back to t . Let f be a function of degree less than k . Before performing the aforementioned handoff, we can let S_1 locally evaluate f over its secret shares. We illustrate this below using multiplication as an example.

Protocol 3 $[x]^{t,q,S_2} \leftarrow \Pi_{h2s}(\llbracket x \rrbracket)$

Input: Public FHE ciphertext $\llbracket x \rrbracket = (a(X), b(X)) \in \mathcal{R}_Q^2$ and the coefficient-wise shared FHE key $[s(X)]^{t,\mathcal{R}_Q,S_1}$ held by S_1

Offline material: $([\tilde{r}(X)]^{t,\mathcal{R}_Q,S_1}, [r]^{t,q,S_2}, [e_{\text{flood}}(X)]^{t,\mathcal{R}_Q,S_1}) \leftarrow \mathcal{F}_{\text{offline}}$

Output: Secret sharing $[x]^{t,q,S_2}$ held by S_2 .

Execution:

- Each $P_i^{(1)}$ computes the ring share $[c(X)]_i = b(X) - a(X)[s(X)]_i - [\tilde{r}(X)]_i + [e_{\text{flood}}(X)]_i$ and sends it to Dealer P_0 ;
- Dealer P_0 reconstructs $c(X) \in \mathcal{R}_Q$, extracts the canonical target coefficient or slot $\bar{c} \in [0, Q)$, and broadcasts $\hat{\delta} = \lfloor \bar{c}/\Delta \rfloor \bmod q$ to S_2 ;
- After receiving $\hat{\delta}$, each $P_i^{(2)}$ in S_2 sets $[x]_i^{t,q,S_2} = [r]_i^{t,q,S_2} + \hat{\delta}$;

Fig. 6. Secure H2S with Handoff Protocol

a) *Case-1 Multiplication:* The multiplication protocol is illustrated in Fig. 5. In the offline phase, committees S_1 and S_2 are provided with correlated randomness $([r]^{2t,S_1}, [r]^{t,S_2})$, respectively, where the former is a degree- $2t$ sharing of r held by S_1 , while the latter is a degree- t sharing of the same value held by S_2 . Given two degree- t sharings $[x]^{t,S_1}$ and $[y]^{t,S_2}$, committee S_1 locally computes $[z]^{2t,S_1} = [x]^{t,S_1} \cdot [y]^{t,S_1} - [r]^{2t,S_1}$, which results in a degree- $2t$ sharing of $z = x \cdot y - r$. All parties in S_1 send their shares of $[\delta]^{2t}$ to the designated dealer P_0 , who reconstructs $\delta = z - r$ and broadcasts it to committee S_2 . Upon receiving δ , the parties in S_2 locally compute $[z]^t = \delta + [r]^t$, thereby obtaining a degree- t sharing of the multiplication result $z = x \cdot y$.

b) *Case-2: Conversion of FHE and SS:* The conversion protocols combine representation conversion with committee handoff by correlating the mask across the representation boundary. In both directions, let $\delta = x - r$ denote the integer difference between the value and its mask. For *FHE-to-SS conversion*, let $\tilde{r}(X) = \Delta \text{Encode}(r) \in \mathcal{R}_Q$. The offline phase provides S_1 with a coefficient-wise ring sharing $[\tilde{r}(X)]^{t,\mathcal{R}_Q,S_1}$ and S_2 with a degree- t sharing $[r]^{t,q,S_2}$ of the corresponding plaintext mask. The two records encode the same hidden random value and satisfy Eq. (23); they are not two independently sampled masks. It also provides S_1 with an independent, one-time sharing $[e_{\text{flood}}(X)]^{t,\mathcal{R}_Q,S_1}$ sampled from the flooding distribution. Given $\llbracket x \rrbracket = (a(X), b(X))$ and the shared FHE secret-key polynomial $[s(X)]^{t,\mathcal{R}_Q,S_1}$, each party in S_1 locally computes a ring share of $c(X) = b(X) - a(X)s(X) - \tilde{r}(X) + e_{\text{flood}}(X)$. The parties send these shares to P_0 , who reconstructs only the masked decryption intermediate and applies the prescribed scaling and rounding to obtain $\hat{\delta} \approx \delta$. After P_0 broadcasts $\hat{\delta}$ to S_2 , every incoming party locally sets $[x]_i^{t,q,S_2} = [r]_i^{t,q,S_2} + \hat{\delta}$. Thus the ciphertext

is converted into a degree- t sharing held by the incoming committee without opening x , as shown in Fig. 6.

The plaintext mask $r\Delta$ hides x but, by itself, does not hide the ciphertext-dependent decryption noise e_x contained in the value opened to P_0 . The independent term e_{flood} statistically smudges this noise. Its distribution is chosen wide enough that $e_x + e_{\text{flood}}$ is statistically insensitive to the admissible shift e_x , while remaining sufficiently smaller than Δ to preserve correct decoding. Neither P_0 nor any coalition of at most t parties learns e_{flood} separately from the reconstructed sum.

For *SS-to-FHE conversion*, the offline phase instead provides S_1 with $[r]^{t,q,S_1}$ and provides a public encryption $\llbracket r \rrbracket = (a(X), b(X)) \in \mathcal{R}_Q^2$ of the same random value. Given $[x]^{t,q,S_1}$, the outgoing parties send shares of $[\delta]^{t,q,S_1} = [x]^{t,q,S_1} - [r]^{t,q,S_1}$ to P_0 . The dealer reconstructs and broadcasts only $\bar{\delta} = \delta \bmod q \in [0, q)$. The incoming committee then performs the public-constant homomorphic addition $\llbracket x \rrbracket = (a(X), b(X) + \Delta \text{Encode}(\bar{\delta}) \bmod Q)$. This produces a public FHE ciphertext while revealing only the one-time-masked difference, as shown in Fig. 7.

Truncation and wrap-around error. The wrap-around is introduced specifically by subtracting the scaled mask $r\Delta$. The following equations apply to one decoded coefficient or slot of the ring relation $u(X) = b(X) - a(X)s(X)$. Let $\bar{u} \in [0, Q)$ be its canonical representative and write $\bar{u} = \Delta x + e_x$. Define the total noise $e_{\text{tot}} = e_x + e_{\text{flood}}$. The modular subtraction reconstructed by the dealer is

$$\begin{aligned} \bar{c} &= (\bar{u} - r\Delta + e_{\text{flood}}) \bmod Q \\ &= \Delta \delta + e_{\text{tot}} + \omega Q, \end{aligned} \quad (13)$$

where ω is the borrow (wrap-around) bit of the subtraction; under the stated one-wrap range, $\omega = \mathbf{1}[\bar{u} + e_{\text{flood}} < r\Delta]$. Consequently, directly truncating the canonical residue gives

$$\hat{\delta} = \left\lfloor \frac{\bar{c}}{\Delta} \right\rfloor = \delta + \left\lfloor \frac{e_{\text{tot}} + \omega Q}{\Delta} \right\rfloor. \quad (14)$$

To characterize the error after conversion to the plaintext field, let $\Delta = \lfloor Q/q \rfloor$ and write $Q = q\Delta + \rho$, where $0 \leq \rho < q$. Reduction modulo q removes the dominant wrap term ωq , leaving

$$\hat{\delta} \bmod q = \delta + \varepsilon \bmod q, \quad \varepsilon = \left\lfloor \frac{e_{\text{tot}} + \omega \rho}{\Delta} \right\rfloor. \quad (15)$$

Thus the conversion error consists of the scaled total noise e_{tot}/Δ , including both the original FHE noise and flooding noise, and the additional wrap-around term $\omega\rho/\Delta$. The latter is small because it contains only the remainder $\rho = Q - q\Delta$, rather than the full modulus Q . In particular, $|\varepsilon| \leq (|e_{\text{tot}}| + \rho)/\Delta + 1/2$; the conversion is exact whenever $|e_{\text{tot}}| + \rho < \Delta/2$. Moreover, if $|e_{\text{tot}}| + \rho < 3\Delta/2$, its magnitude is at most one plaintext unit. If $Q = q\Delta$, then $\rho = 0$ and wrap-around contributes no additional error at all. The resulting sharing represents $x + \varepsilon \bmod q$.

For S2H, the subtraction $x - r$ may also wrap around in the SS field. For canonical $x, r \in [0, q)$, write

$$\bar{\delta} = \delta \bmod q = \delta + \omega q, \quad \omega = \mathbf{1}[x < r]. \quad (16)$$

Protocol 4 $\llbracket x \rrbracket \leftarrow \Pi_{\text{S2H}}([x]^{t,q,S_1})$

Input: Secret share $[x]^{t,q,S_1}$ held by S_1 .

Offline material: $([r]^{t,q,S_1}, \llbracket r \rrbracket = (a(X), b(X))) \in \mathcal{R}_Q^2 \leftarrow \mathcal{F}_{\text{offline}}$

Output: Public FHE ciphertext $\llbracket x \rrbracket$.

Execution:

- Each party $P_i^{(1)}$ in S_1 calculates $[\delta]_i = [x]_i^{S_1} - [r]_i^{S_1}$ and sends $[\delta]_i$ to Dealer P_0 ;
- Dealer P_0 reconstructs the canonical value $\bar{\delta} \in [0, q)$ by Lagrange interpolation and broadcasts $\bar{\delta}$ to S_2 ;
- After receiving $\bar{\delta}$, each $P_i^{(2)}$ in S_2 sets $\llbracket x \rrbracket = (a(X), b(X) + \Delta \text{Encode}(\bar{\delta}) \bmod Q)$;

Fig. 7. Secure S2H with Handoff Protocol

Let the offline ciphertext of r decrypt to $\Delta r + e_r \bmod Q$. After adding $\bar{\delta}\Delta$ and using $q\Delta = Q - \rho$, the output ciphertext decrypts to

$$\Delta r + e_r + \bar{\delta}\Delta = \Delta x + e_r - \omega\rho \pmod{Q}. \quad (17)$$

Hence the large term $\omega q\Delta$ disappears modulo Q ; the wrap-around changes only the effective FHE noise from e_r to $e_r - \omega\rho$. Equivalently, the S2H conversion error is $\varepsilon = \lfloor (e_r - \omega\rho)/\Delta \rfloor$, and decoding returns $x + \varepsilon \bmod q$. A sufficient condition for an exact result is $|e_r| + \rho < \Delta/2$, while $|e_r| + \rho < 3\Delta/2$ bounds the decoding error to at most one plaintext unit. Similarly, when $Q = q\Delta$, $\rho = 0$ and the wrap-around of $x - r$ has no effect on the output ciphertext; only the original FHE error e_r remains.

When a conversion crosses the authenticated MPC boundary, the runtime uses the MAC-aware variants rather than the unauthenticated interfaces above. Authenticated H2S correlates the ring mask with an authenticated sharing of the same r and outputs $\langle\langle x \rangle\rangle$, while authenticated S2H verifies the MAC of the opened difference before modifying $\llbracket r \rrbracket$. The full flows are given in Protocols 9 and 10 in Appendix A.

Dealing with bit representation. When an MPC operator requires the bit representation of a field-shared value, the runtime additionally invokes the arithmetic-to-Boolean conversion Π_{S2B} . This conversion opens only a one-time-masked field element and reconstructs the canonical bits through a Boolean addition and conditional modular reduction. Its complete protocol is provided as Protocol 11 in Appendix A. The Boolean wires are Shamir-shared over $\text{GF}(2^\kappa)$, with $2^\kappa - 1 \geq n$, rather than over $\mathbb{F}_2$.

c) Case-3: Authentication: Note (MAC generation). MAC-tag shares are generated by the committee with the ordinary secret-sharing multiplication protocol; they do not require a separate MAC-generation primitive. Given $[x]^{t,q,S}$ and the shared MAC key $[\alpha]^{t,q,S}$, the committee invokes

$$[\gamma_x]^{t,q,S} \leftarrow \Pi_{\text{mult}}([x]^{t,q,S}, [\alpha]^{t,q,S}), \quad \gamma_x = x\alpha. \quad (18)$$

The multiplication protocol performs the required degree reduction from $2t$ back to t . When MAC generation is combined with committee rotation, the multiplication-with-handoff variant places $[\gamma_x]$ directly in the incoming committee. Thus the term “authenticated sharing” denotes the pair $([x], [\gamma_x])$ after this ordinary multiplication has completed.

Authentication is pipelined across two consecutive handoffs. Let S_1 , S_2 , and S_3 be three consecutive committees, and let α be the secret-shared MAC key. For a value x , the outgoing committee S_1 holds both its value sharing $[x]^{t,S_1}$ and its MAC-tag sharing $[x\alpha]^{t,S_1}$. During the handoff from S_1 to S_2 , the two sharings are transferred independently using two one-time correlated masks. The dealer reconstructs only the masked differences, and S_2 obtains candidate sharings $[x]^{t,S_2}$ and $[x\alpha]^{t,S_2}$. At this point the transferred state is provisional: its MAC relation is checked during the following handoff rather than inside the same S_1 -to- S_2 transfer.

When execution advances from S_2 to S_3 , committees S_2 and S_3 invoke the multiplication-with-handoff protocol on $[x]^{t,S_2}$ and $[x\alpha]^{t,S_2}$. This produces in S_3 a fresh sharing $[z]^{t,S_3}$ of $z = x\alpha$. In parallel, the claimed tag $[x\alpha]^{t,S_2}$ is handed off to S_3 . The new committee then computes

$$[v]^{2t,S_3} = ([z]^{t,S_3} - [x\alpha]^{t,S_3})[r]^{t,S_3}, \quad (19)$$

for a fresh random checking value r , and opens the resulting degree- $2t$ sharing (requiring $n > 2t$). If $v = 0$, the tag transferred from S_1 is consistent with the transferred value and MAC key, so S_3 accepts the chained handoff. Otherwise, it rejects the provisional state. Consequently, the handoff from S_1 to S_2 is authenticated as part of the subsequent S_2 -to- S_3 handoff, as specified in Protocol 7 in Appendix A.

The same pipeline applies to multiplication. Committee S_1 transfers both the multiplication result $z = xy$ and its claimed tag $z\alpha$ to S_2 using degree-reducing correlated masks. During the next handoff, S_2 and S_3 recompute $z\alpha$ through Protocol 8, compare it with the handed-off tag using the random check above, and accept only if the opened check is zero. No provisional value may be opened externally or committed as final output before this deferred authentication succeeds. The complete authenticated multiplication flow is given in Appendix A.

Representation conversion uses the same MAC invariant. In authenticated H2S, the destination mask already carries its tag, so the public masked difference updates both the value share and MAC share linearly. In authenticated S2H, the committee checks the MAC of the masked difference before using it in the FHE ciphertext. These conversion-specific constructions are specified in Protocols 9 and 10, respectively.

B. Shared Offline Preprocessing

The offline phase generates data-independent material before private inputs are known. Every item is tagged by epoch, consumer, and a unique one-time identifier. For epoch ℓ , the runtime partitions the pool as

$$\begin{aligned} \text{Prep}^\ell = & \text{Prep}_{\text{HO}}^\ell \uplus \text{Prep}_{\text{H2S}}^\ell \uplus \text{Prep}_{\text{S2H}}^\ell \\ & \uplus \text{Prep}_{\text{Mul}}^\ell \uplus \text{Prep}_{\text{Pack}}^\ell \uplus \text{Prep}_{\text{MAC}}^\ell. \end{aligned} \quad (20)$$

Protocol 5 $[r_{\text{sh}}]^{t,q,S} \leftarrow \Pi_{\text{RandShare}}^t$

Correlated Randomness: For every $A \in \mathcal{H}_t$, the parties in A hold a common PRSS seed κ_A that is unknown to parties outside A .

Output: A fresh degree- t Shamir sharing $[r]^{t,q,S}$.

Execution:

- Every party $P_i \in A$ generate randomness $r_A = \text{PRF}_{\kappa_A}(\tau) \in \mathbb{F}_q$;
- Setting public polynomial $u_A^{(d)}(X) = \prod_{j \notin A} \frac{X - \alpha_j}{-\alpha_j}$, each party P_i locally calculates

$$[r]_i^{t,q,S} = \sum_{A \in \mathcal{H}_t} r_A u_A^{(t)}(\alpha_i).$$

The handoff protocol consumes a correlated sharing pair generated by $\Pi_{\text{ResharePair}}^{t_1,t_2}$ from Prep_{HO} ; H2S consumes scheme-specific decoding material; S2H consumes a correlated sharing/ciphertext pair; scalar and packed secret multiplication consume the outputs of Π_{PairGen} and $\Pi_{\text{Pair}}^{\text{pack}}$, respectively. Public linear operations consume no correlated material. Authenticated opening uses the current MAC state and a fresh public batch challenge sampled only after the values are fixed. At handoff, old-epoch items are consumed or invalidated and the incoming committee generates a fresh partition under its epoch identifier.

1) *PRSS and RandShare:* The preprocessing protocols use the prime field $\mathbb{F}_q$ fixed for Shamir sharing. Every nonzero evaluation point and every nonzero difference used by the interpolation bases below is invertible. Let $\mathcal{S} = \{P_1, \dots, P_n\}$ and let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q \setminus \{0\}$ be distinct public Shamir evaluation points, so $q > n$. For $0 \leq d < n$, define $\mathcal{H}_d = \{A \subseteq [n] : |A| = n - d\}$. Each $A \in \mathcal{H}_d$ is associated with the public polynomial

$$u_A^{(d)}(X) = \prod_{j \notin A} \frac{X - \alpha_j}{-\alpha_j}. \quad (21)$$

It has degree d , satisfies $u_A^{(d)}(0) = 1$, and $u_A^{(d)}(\alpha_i) = 0$ at α_i for every $i \notin A$. A correlated randomness r_A known by the parties in A becomes a valid degree- d Shamir secret share $[r_A]^t$ after multiplication by $u_A^{(d)}(X)$, since parties not in A . For any party $P_i \notin A$, which does not know r_A , its resulting share remains zero after multiplication by r_A .

Following PRSS techniques [15], offline preprocessing gives each holder set an independent common random value. For every coalition C of at most t parties, at least one required random component is unknown to all parties in C .

The outputs are evaluations of the single polynomial $F_{r_{\text{sh}}}(X) = \sum_{A \in \mathcal{H}_t} r_A u_A^{(t)}(X)$, with $F_{r_{\text{sh}}}(0) = r_{\text{sh}}$ and $F_{r_{\text{sh}}}(\alpha_i) = [r_{\text{sh}}]_i$. For any coalition of at most t parties, at least one holder set is disjoint from the coalition, so its component is unknown to that coalition. Protocol 5 is a base generator

Protocol 6 $([r]^{t_1,q,S_1}, [r]^{t_2,q,S_2}) \leftarrow \Pi_{\text{ResharePair}}^{t_1,t_2}$

Correlated Randomness: For every $(A_i^{(1)}, A_j^{(2)}) \in \mathcal{H}_{t_1}^{(1)} \times \mathcal{H}_{t_2}^{(2)}$, all parties in the two holder groups $A_i^{(1)}$ and $A_j^{(2)}$ hold a common PRSS seed $\kappa_{i,j}$ that is unknown outside their union.

Output: A degree- t_1 Shamir sharing $[r]^{t_1,q,S_1}$ and a degree- t_2 Shamir sharing $[r]^{t_2,q,S_2}$ of the same fresh random value $r \in \mathbb{F}_q$.

Execution:

- The parties in $A_i^{(1)} \in \mathcal{S}_1$ and $A_j^{(2)} \in \mathcal{S}_2$ generate same randomness $\zeta_{i,j} = \text{PRF}_{\kappa_{i,j}}(\tau) \in \mathbb{F}_q$ for every $(A_i^{(1)}, A_j^{(2)}) \in \mathcal{H}_{t_1}^{(1)} \times \mathcal{H}_{t_2}^{(2)}$.
- Each holder group in $\mathcal{S}_1$ computes its row aggregate, while each holder group in $\mathcal{S}_2$ computes its column aggregate:

$$r_{A_i}^{(1)} = \sum_{A_j^{(2)} \in \mathcal{H}_{t_2}^{(2)}} \zeta_{i,j}, \quad r_{A_j}^{(2)} = \sum_{A_i^{(1)} \in \mathcal{H}_{t_1}^{(1)}} \zeta_{i,j}.$$

- Each $P_k^{(1)} \in \mathcal{S}_1$ and $P_k^{(2)} \in \mathcal{S}_2$ locally outputs, respectively,

$$[r]_k^{t_1,q,S_1} = \sum_{\substack{A_i^{(1)} \in \mathcal{H}_{t_1}^{(1)} \\ k \in A_i^{(1)}}} r_{A_i}^{(1)} u_{A_i}^{(t_1,1)}(\alpha_k^{(1)}),$$

$$[r]_k^{t_2,q,S_2} = \sum_{\substack{A_j^{(2)} \in \mathcal{H}_{t_2}^{(2)} \\ k \in A_j^{(2)}}} r_{A_j}^{(2)} u_{A_j}^{(t_2,2)}(\alpha_k^{(2)}).$$

rather than an online operator. Its explicit caller is Π_{AuthRand} ; a concrete H2S preprocessing protocol may also call it when its required output relation has been specified.

2) *Handoff Re-sharing Randomness:* Let $\mathcal{S}_1 = \{P_1^{(1)}, \dots, P_n^{(1)}\}$ and $\mathcal{S}_2 = \{P_1^{(2)}, \dots, P_n^{(2)}\}$ be the outgoing and incoming committees with respective corruption thresholds t_1 and t_2 . For $b \in \{1, 2\}$, define $\mathcal{H}_{t_b}^{(b)} = \{A^{(b)} \subseteq [n] : |A^{(b)}| = n - t_b\}$ and let $u_A^{(t_b,b)}$ denote the holder polynomial in Eq. (21), evaluated at the public Shamir points of $\mathcal{S}_b$. The protocol below constructs $\text{Prep}_{\text{HO}}^\ell := ([r]^{t_1,q,S_1}, [r]^{t_2,q,S_2})$ over $\mathbb{F}_q$ without reconstructing r .

Indeed, the two sharing polynomials have the same constant term because

$$\sum_{A_i^{(1)} \in \mathcal{H}_{t_1}^{(1)}} r_{A_i}^{(1)} = \sum_{A_i^{(1)} \in \mathcal{H}_{t_1}^{(1)}} \sum_{A_j^{(2)} \in \mathcal{H}_{t_2}^{(2)}} \zeta_{i,j} = \sum_{A_j^{(2)} \in \mathcal{H}_{t_2}^{(2)}} r_{A_j}^{(2)} = r. \quad (22)$$

For any coalitions $C_1 \subseteq \mathcal{S}_1$ and $C_2 \subseteq \mathcal{S}_2$ with $|C_1| \leq t_1$ and $|C_2| \leq t_2$, there exist holder groups disjoint from C_1 and C_2 , respectively. The corresponding $\zeta_{i,j}$ is therefore unknown to the joint coalition. Each pair is tagged with its epoch and

one-time derivation label and is consumed once by Protocol 1.

3) *Multiplication*: The offline pair required by the multiplication protocol can be generated by invoking $\Pi_{\text{ResharePair}}^{2t,t}$.

4) *Conversion*: For H2S, the cross-domain preprocessing functionality samples $r \in \mathbb{F}_q$ once and distributes $[r]^{t,q,S_2}$ together with a coefficient-wise ring sharing $[\tilde{r}(X)]^{t,\mathcal{R}_Q,S_1}$ of $\tilde{r}(X) = \Delta \text{Encode}(r)$. These are sharings in different algebraic domains and are not identified with one another: the first is a scalar Shamir sharing over $\mathbb{F}_q$, whereas the second shares an encoded polynomial in $\mathcal{R}_Q$. Their required correlation is

$$\text{Decode}(\tilde{r}(X)) = r, \quad \tilde{r}(X) = \Delta \text{Encode}(r) \pmod{Q}. \quad (23)$$

Neither committee can choose its component independently, and the pair is tagged with a common conversion identifier and consumed atomically.

Cross-domain generation. We realize this correlation from bit-decomposed preprocessing rather than by lifting individual field shares. Let $\ell = \lceil \log_2 q \rceil$. The offline backend first generates correlated random-bit sharings

$$([\beta_j]^{t,q,S_2}, [\beta_j]^{t,\mathcal{R}_Q,S_1}), \quad \begin{array}{l} \beta_j \in \{0, 1\}, \\ 0 \leq j < \ell, \end{array} \quad (24)$$

using the actively secure preprocessing backend. The backend verifies Booleanity and equality of the two representations before releasing either sharing; in the ring-domain component, $[\beta_j]^{t,\mathcal{R}_Q,S_1}$ denotes a sharing of $\text{Encode}(\beta_j)$. The parties locally form

$$\begin{aligned} [r]^{t,q,S_2} &= \sum_{j=0}^{\ell-1} 2^j [\beta_j]^{t,q,S_2}, \\ [\tilde{r}(X)]^{t,\mathcal{R}_Q,S_1} &= \Delta \sum_{j=0}^{\ell-1} 2^j [\beta_j]^{t,\mathcal{R}_Q,S_1}. \end{aligned} \quad (25)$$

A secure range check accepts only candidates with $r = \sum_j 2^j \beta_j < q$; rejected candidates are discarded before use. Hence the accepted r is uniform in $\mathbb{F}_q$, and the ring sharing encodes its same canonical integer representative. This range check is important when q is not a power of two.

A tempting alternative is to generate $[r]^{t,q}$ and independently lift each party's share into $\mathbb{Z}_Q$ before multiplying by Δ . This is not correct in general: Shamir shares are reduced modulo q , and their interpolation may contain a multiple of q ; after lifting, that multiple becomes $q\Delta = Q - \rho$, which is nonzero modulo Q whenever $\rho = Q - q\Delta \neq 0$. Bit-decomposed generation avoids this cross-modulus carry and guarantees Eq. (23).

For each H2S invocation, preprocessing additionally generates an independent coefficient-wise sharing $[e_{\text{flood}}(X)]^{t,\mathcal{R}_Q,S_1}$ of a signed flooding polynomial in $\mathcal{R}_Q$. The sample is bound to one conversion identifier and consumed once. Its width must dominate the maximum admissible input-ciphertext noise for statistical smudging, while the combined bound on $e_x + e_{\text{flood}}$ must satisfy the decoding condition derived above. The flooding sharing is added before reconstruction, so P_0 observes only the flooded masked-decryption intermediate.

For the S2H pair $([r]^{t,q,S_1}, \llbracket r \rrbracket = (a(X), b(X)) \in \mathcal{R}_Q^2)$, the plaintext mask is sampled in $\mathbb{F}_q$. Specifically, we first invoke $[r_{\text{sh}}]^{t,q,S_1} \leftarrow \Pi_{\text{RandShare}}^t$, and then homomorphically encrypt and publicly release each intermediate value r_A . These ciphertexts are subsequently aggregated homomorphically to obtain an encryption of $r = \sum_A r_A$.

5) *Authentication*: The offline pair required by the authentication protocol can be generated by invoking $\Pi_{\text{ResharePair}}^{t,t}$ twice.

IV. IMPLEMENTATION AND BENCHMARK

This section is structured around five evaluation questions. *RQ1* asks which cryptographic operators dominate online latency and communication. *RQ2* asks how network conditions, committee size, and packing affect performance. *RQ3* asks how these costs compose in complete confidential contract invocations. *RQ4* asks how the number of live private values affects committee rotation. *RQ5* asks whether hybrid execution reduces workload cost and whether representation conversion preserves numerical correctness under the selected FHE parameters. We report latency, throughput, bytes transmitted, and the split between offline preprocessing and online execution. Unless stated otherwise, latency should be reported as the median and interquartile range over repeated trials after warm-up; throughput should be measured at steady state with a fixed concurrency level.

A. Implementation and Experimental Setup

The prototype is implemented in Rust (rustc 1.94.1) and uses OpenFHE (BFVRNS) as the FHE backend, linked through a C ABI wrapper built with MinGW64 g++. MPC arithmetic runs over $\mathbb{F}_r$, the 254-bit BLS12-381 scalar field (`ark-bn254::Fr`), with boolean shares in $GF(2^8)$; the FHE layer uses BFV with plaintext modulus $p = 65537$, batch size 8, and multiplicative depth 2. Measurements were taken on a 12th Gen Intel Core i7-12700H (14 cores, 20 threads, 2.3 GHz) running Windows 11.

We evaluate committee sizes $n \in \{4, 8, 16, 32, 64\}$ under the honest-majority threshold $t = \lfloor (n-1)/2 \rfloor$. LAN, MAN, and WAN profiles are defined by their bandwidth and RTT: LAN 0.5 ms RTT / 1 Gbps, MAN 20 ms RTT / 100 Mbps, and WAN 100 ms RTT / 10 Mbps. Each profile is enforced between every pair of committee nodes and validated using independent bandwidth and RTT measurements before each experiment batch. Unless otherwise noted, we summarize latency using the median and IQR and report the total bytes transmitted per operation.

B. Protocol Microbenchmarks

To answer RQ1, we benchmark SS handoff, authenticated multiplication with degree reduction, H2S, and S2H independently. Each trial consumes fresh preprocessing material and starts from already distributed inputs. We measure online wall-clock latency from protocol dispatch until all destination parties accept the output, sum sent bytes across parties, and separately record offline material generation. Local encoding,

TABLE I
MICROBENCHMARK ONLINE LATENCY UNDER THE LAN PROFILE
($n = 16$, $t = 7$; MEDIAN OVER REPEATED TRIALS).

Operation	Online (ms)
SS handoff	1.90
SS multiplication	1.92
H2S conversion	106.66
S2H conversion	14.08

encryption, and serialization costs are reported as separate subcomponents rather than hidden in network time.

To answer RQ2, the same operation is repeated while varying exactly one factor: committee size, network profile, protected-state width, or packing factor. The remaining parameters are held fixed and printed with the corresponding result. Figure 8 illustrates the intended committee-size comparison separately under LAN, MAN, and WAN. The final figure should add uncertainty bars from the raw trials.

C. End-to-End Contract Workloads

To answer RQ3, we use three operator mixes rather than a single synthetic TPS loop: (i) *private transfer*, dominated by authenticated arithmetic and a comparison; (ii) *sealed-bid selection*, which exercises conversion and MPC comparison; and (iii) *batched analytics*, which emphasizes packed FHE arithmetic followed by one conversion. For each workload, the contract, input dimensions, operator trace, number of representation changes, output representation, and committee-rotation schedule must be fixed before running the experiment. We report per-invocation latency and sustained throughput, and include separate static-committee and rotation-enabled runs to isolate handoff overhead.

D. Committee-Rotation Scalability

To answer RQ4, we trigger an epoch transition with $m \in \{1, 16, 64, 256, 1024\}$ live SS records while fixing the record width, committee size, and threshold. The timer starts when the outgoing committee receives the finalized next-committee roster and ends when the incoming committee has verified and installed every transferred record. We report total rotation latency, total traffic, and amortized latency per record. Public FHE ciphertexts are excluded because they can be copied without resharing; the SS record holding the FHE secret key is included. A companion run varies the epoch length to report the fraction of execution time consumed by rotation.

E. Hybrid Ablation and Conversion Accuracy

To answer the first part of RQ5, each end-to-end workload is executed with the same semantics and security target under three operator-placement policies: all eligible operators in SS, all eligible operators in FHE, and the hybrid placement used by PrivFlux. Setup and preprocessing are reported separately, and unsupported baseline operations are identified rather than silently omitted. Table II reports the measured latency.

TABLE II
END-TO-END LATENCY ABLATION IN MILLISECONDS (LAN, $n = 16$, $t = 7$; MEDIAN OVER REPEATED TRIALS). THE ALL-FHE PLACEMENT CANNOT EVALUATE THE COMPARISON OPERATORS (BFV HAS NO COMPARISON), SO ITS WORKLOADS ARE MARKED N/A; ITS LOCAL ARITHMETIC (BFV EvalAdd) COSTS ≈ 0.13 MS.

Placement	Transfer	Selection	Analytics
All SS	47.99	133.68	41.77
All FHE	N/A	N/A	N/A
Hybrid (PrivFlux)	164.85	241.88	148.91

TABLE III
CONVERSION-ERROR SWEEP (NUMERICAL SIMULATION OF THE TRUNCATION-AND-WRAP ERROR MODEL IN SECTION III; UNIFORM NORMALIZED NOISE). “MISMATCH” IS THE FRACTION OF OUTPUTS DIFFERING FROM x IN $\mathbb{F}_q$.

Noise/ Δ	ρ/Δ	Trials	H2S (%)	S2H (%)
0.05	0.00	100000	0.000	0.000
0.20	0.10	100000	0.000	0.000
0.40	0.15	100000	3.070	3.098
0.55	0.20	100000	15.991	15.756

To answer the numerical part of RQ5, we sample canonical $x, r \in [0, q)$ and exercise both conversion directions while sweeping the normalized initial total H2S noise $|e_x + e_{\text{flood}}|/\Delta$ (or S2H mask-ciphertext noise $|e_r|/\Delta$) and modulus remainder ρ/Δ , where $Q = q\Delta + \rho$. For each parameter point, the final artifact will report the number of trials, the exact-decoding rate, the distribution of signed plaintext error, and results partitioned by whether the subtraction wraps. This directly tests the direction-specific error ε derived for H2S and S2H in Section III. Table III reports the mismatch rate from a numerical simulation of this model.

F. Baselines and Reporting Discipline

The final evaluation should compare implementations with the same contract semantics and security parameters: an all-FHE design with its key-management path, a static hybrid FHE-MPC design, and PrivFlux with committee rotation. For each comparison, we will disclose unsupported operators, setup exclusions, and whether key setup and preprocessing are amortized. No speedup or scalability claim will be made from the measurements above. Raw per-trial observations, plot-generation inputs, and the exact commands used to reproduce each figure should accompany the artifact.

V. CONCLUSION

REFERENCES

- [1] A. Banerjee, M. Clear, and H. Tewari, “zkHawk: Practical private smart contracts from MPC-based Hawk,” in *Proc. 3rd Conference on Blockchain Research & Applications for Innovative Networks and Services*, 2021, pp. 245–248.
- [2] C. Baum, J. H.-y. Chiang, B. David, and T. K. Frederiksen, “Eagle: Efficient privacy preserving smart contracts,” in *Proc. Financial Cryptography and Data Security*, 2023.
- [3] B. Bünz, S. Agrawal, M. Zamani, and D. Boneh, “Zether: Towards privacy in a smart contract world,” in *Proc. Financial Cryptography and Data Security*, 2020, pp. 423–443.

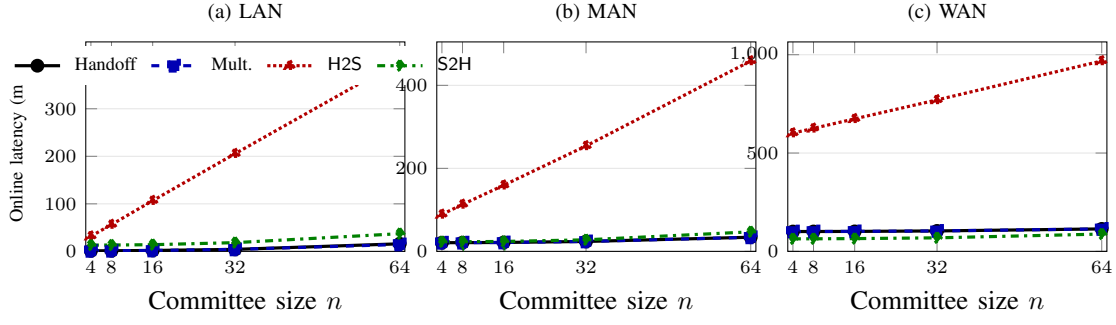


Fig. 8. Online latency of the protocol microbenchmarks under (a) LAN, (b) MAN, and (c) WAN ($t = \lfloor (n-1)/2 \rfloor$; median over repeated trials).

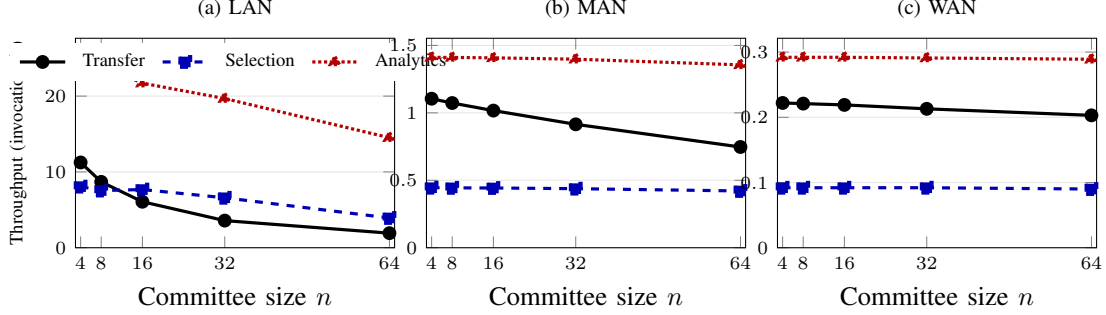


Fig. 9. End-to-end throughput under (a) LAN, (b) MAN, and (c) WAN (single-concurrency, i.e. $1/\text{latency}$; $t = \lfloor (n-1)/2 \rfloor$).

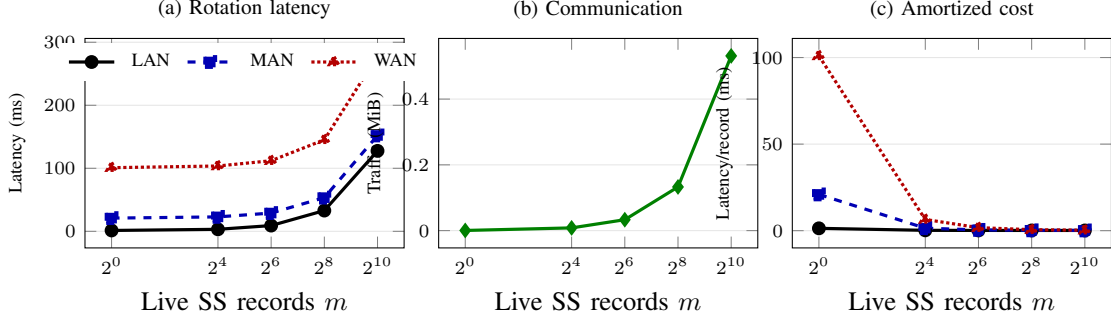


Fig. 10. Scaling of an epoch handoff with the number of live SS records: (a) end-to-end rotation latency, (b) aggregate communication, and (c) amortized latency per record ($n = 16$, $t = 7$; median over repeated trials).

- [4] S. Steffen, B. Bichsel, M. Gersbach, N. Melchior, P. Tsankov, and M. Vechev, “zkay: Specifying and enforcing data privacy in smart contracts,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1759–1776.
- [5] R. Solomon, R. Weber, and G. Almashaqbeh, “smartFHE: Privacy-preserving smart contracts from fully homomorphic encryption,” in *Proc. IEEE European Symposium on Security and Privacy*, 2023, pp. 309–331.
- [6] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, “Zerocash: Decentralized anonymous payments from Bitcoin,” in *Proc. IEEE Symposium on Security and Privacy*, 2014, pp. 459–474.
- [7] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, and H. Wu, “Zexe: Enabling decentralized private computation,” in *Proc. IEEE Symposium on Security and Privacy*, 2020, pp. 820–837.
- [8] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou, “Hawk: The blockchain model of cryptography and privacy-preserving smart contracts,” in *Proc. IEEE Symposium on Security and Privacy*, 2016, pp. 839–858.
- [9] G. Kapshuk, M. Green, and I. Miers, “Giving state to the stateless: Augmenting trustworthy computation with ledgers,” in *Proc. Network and Distributed System Security Symposium*, 2019.
- [10] A. Banerjee and H. Tewari, “Multiverse of HawkNess: A universally-composable MPC-based Hawk variant,” *Cryptography*, vol. 6, no. 3, 2022, art. 39.
- [11] Zama, “fhevm: A cross-chain protocol for confidential smart contracts,” Technical Report, Version 3.1, Jun. 2025.
- [12] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [13] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, “Multiparty computation from somewhat homomorphic encryption,” in *Advances in Cryptology—CRYPTO*, ser. LNCS, vol. 7417, 2012, pp. 643–662.
- [14] V. Goyal and Y. Song, “Malicious security comes free in honest-majority MPC,” *Cryptology ePrint Archive*, Tech. Rep. 2020/134, 2020.
- [15] R. Cramer, I. Damgård, and Y. Ishai, “Share conversion, pseudorandom secret-sharing and applications to secure computation,” in *Theory of Cryptography Conference—TCC*, ser. LNCS, vol. 3378, 2005, pp. 342–362.
- [16] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, and A. Sahai, “Threshold cryptosystems from threshold fully homomorphic encryption,” in *Advances in Cryptology—CRYPTO*,

2018, pp. 565–596.

- [17] M. Dahl, D. Demmler, S. El Kazdadi, A. Meyre, J.-B. Orfila, D. Rotaru, N. P. Smart, S. Tap, and M. Walter, “Noah’s Ark: Efficient threshold-FHE using noise flooding,” Cryptology ePrint Archive, Tech. Rep. 2023/815, 2023.
- [18] A. Herzberg, S. Jarecki, H. Krawczyk, and M. Yung, “Proactive secret sharing or: How to cope with perpetual leakage,” in *Advances in Cryptology—CRYPTO*, 1995, pp. 339–352.
- [19] A. R. Choudhuri, A. Goel, M. Green, A. Jain, and G. Kaptchuk, “Fluid MPC: Secure multiparty computation with dynamic participants,” in *Advances in Cryptology—CRYPTO, Part II*, 2021, pp. 94–123.

APPENDIX

This appendix gives the complete protocol flows for the deferred authentication and share-to-bit conversion mechanisms used by the runtime. The first four protocols augment handoff, multiplication, H2S, and S2H with MAC-protected shares, whereas the final protocol converts an arithmetic sharing into Boolean sharings without opening the underlying value. Every inter-committee protocol uses the finalization rule of Section III-A: after the destination committee accepts and certifies the new state, honest source parties securely erase obsolete shares and epoch-specific secret material while retaining the public transcript.

A. Authenticated Handoff

Protocol 7 transfers both a value sharing and its claimed MAC-tag sharing from $\mathcal{S}_1$ to $\mathcal{S}_2$. Two independent correlated masks are required because the value x and the tag $x\alpha$ are distinct secrets. The outgoing committee opens only the masked differences $\delta_1 = x - r_1$ and $\delta_2 = x\alpha - r_2$ to the dealer; adding these public differences to the corresponding destination-side masks gives fresh sharings of the same value and tag in $\mathcal{S}_2$.

The transferred pair remains provisional until the next handoff. During that handoff, the parties recompute $x\alpha$ from the transferred value and the shared MAC key, transfer the claimed tag to $\mathcal{S}_3$, and open a fresh random multiple of their difference. A zero result establishes the required MAC relation for this check, whereas a nonzero result causes rejection. The fresh checking value and both correlated mask pairs must be used only once.

B. Authenticated Multiplication

Protocol 8 combines multiplication, degree reduction, committee handoff, and MAC-tag propagation. Local multiplication in $\mathcal{S}_1$ produces degree- $2t$ sharings of $z = xy$ and $z\alpha = (x\alpha)y$. The source-side masks therefore also have degree $2t$, while their correlated destination-side sharings have degree t . After the dealer reconstructs the two masked differences, $\mathcal{S}_2$ adds them to its masks and obtains degree- t candidate sharings of both z and $z\alpha$.

As with authenticated handoff, correctness of the MAC relation is checked in the following handoff. The parties recompute $z\alpha$, compare it with the transferred tag, multiply the difference by a fresh random checking value, and open the result. Until this check succeeds, neither the candidate product nor its tag is released as an externally visible output.

Protocol 7 $[x]^{S_2} \leftarrow \Pi_{\text{auth-handoff}}([x]^{S_1})$

Input: Secret share $[x]^{S_1}$ and $[x \cdot \alpha]^{S_1}$ held by $\mathcal{S}_1$.

Offline material: $([r_1]^{S_1}, [r_1]^{S_2}, [r_2]^{S_1}, [r_2]^{S_2}) \leftarrow \mathcal{F}_{\text{offline}}$

Output: Secret share $([x]^{S_2}, [x \cdot \alpha]^{S_2})$ held by $\mathcal{S}_2$.

Execution:

- Each party $P_i^{(1)}$ in $\mathcal{S}_1$ calculates $[\delta_1]_i = [x]_i^{S_1} - [r_1]_i^{S_1}$, $[\delta_2]_i = [x \cdot \alpha]_i^{S_1} - [r_2]_i^{S_1}$ and sends $[\delta_1]_i$, $[\delta_2]_i$ to Dealer P_0 ;
- Dealer P_0 reconstruct δ_1 and δ_2 by Lagrange interpolation and broadcast them to $\mathcal{S}_2$;
- After receiving δ_1 and δ_2 , each $P_i^{(2)}$ in $\mathcal{S}_2$ sets $[x]_i^{S_2} = [r_1]_i^{S_2} + \delta_1$, $[x \cdot \alpha]_i^{S_2} = [r_2]_i^{S_2} + \delta_2$;

Authentication:

- $\mathcal{S}_2$ and $\mathcal{S}_3$ invoke $[z]^{t, S_3} \leftarrow \Pi_{\text{mult}}([x]^{t, S_2}, [\alpha]^{t, S_2})$;
- Each party $P_i^{(1)}$ in $\mathcal{S}_3$ calculates $[v]_i = ([z]_i^{t, S_3} - [x \cdot \alpha]_i^{t, S_3}) \cdot [r]_i^{t, S_3}$;
- Open v and check if it equals 0.

Fig. 11. Secure Share Handoff Protocol with Authentication

Protocol 8 $[z]^{S_2} \leftarrow \Pi_{\text{auth-mult}}([x]^{S_1}, [y]^{S_1})$

Input: Secret share $([x]^{S_1}, [x \cdot \alpha]^{S_1})$, $[y]^{S_1}$ held by $\mathcal{S}_1$.

Offline material: $([r_1]^{2t, S_1}, [r_1]^{t, S_2}, [r_2]^{2t, S_1}, [r_2]^{t, S_2}) \leftarrow \mathcal{F}_{\text{offline}}$

Output: Secret share $([z]^{S_2}, [z \cdot \alpha]^{S_2})$ held by $\mathcal{S}_2$.

Execution:

- Each party $P_i^{(1)}$ in $\mathcal{S}_1$ calculates $[\delta_1]_i = [x]_i^{S_1} \cdot [y]_i^{S_1} - [r_1]_i^{S_1}$, $[\delta_2]_i = [x \cdot \alpha]_i^{S_1} \cdot [y]_i^{S_1} - [r_2]_i^{S_1}$ and sends $[\delta_1]_i$, $[\delta_2]_i$ to Dealer P_0 ;
- Dealer P_0 reconstruct δ_1 and δ_2 by Lagrange interpolation and broadcast them to $\mathcal{S}_2$;
- After receiving δ_1 and δ_2 , each $P_i^{(2)}$ in $\mathcal{S}_2$ sets $[z]_i^{S_2} = [r_1]_i^{S_2} + \delta_1$, $[z \cdot \alpha]_i^{S_2} = [r_2]_i^{S_2} + \delta_2$;

Authentication:

- $\mathcal{S}_2$ and $\mathcal{S}_3$ invoke $[\gamma]^{t, S_3} \leftarrow \Pi_{\text{mult}}([z]^{t, S_2}, [\alpha]^{t, S_2})$;
- Each party $P_i^{(1)}$ in $\mathcal{S}_3$ calculates $[v]_i = ([\gamma]_i^{t, S_3} - [z \cdot \alpha]_i^{t, S_3}) \cdot [r]_i^{t, S_3}$;
- Open v and check if it equals 0.

Fig. 12. Secure Multiplication Protocol with Authentication

C. Authenticated FHE-to-SS Conversion

Protocol 9 augments H2S with an authenticated destination mask. The cross-domain offline item contains the same random $r \in \mathbb{F}_q$ as a scaled ring sharing $[\tilde{r}(X)]^{t, \mathcal{R}_Q, S_1}$ and as an authenticated field sharing $\langle\langle r \rangle\rangle^{t, q, S_2} = ([r]^{t, q, S_2}, [\gamma_r]^{t, q, S_2})$,

Protocol 9 $\langle\langle x \rangle\rangle^{t,q,S_2} \leftarrow \Pi_{\text{auth-h2s}}(\llbracket x \rrbracket)$

Input: Public ciphertext $\llbracket x \rrbracket = (a(X), b(X)) \in \mathcal{R}_Q^2$; $[s(X)]^{t,\mathcal{R}_Q,S_1}$; and $[\alpha]^{t,q,S_2}$.

Offline material: $([\tilde{r}(X)]^{t,\mathcal{R}_Q,S_1}, \langle\langle r \rangle\rangle^{t,q,S_2}, [e_{\text{flood}}(X)]^{t,\mathcal{R}_Q,S_1})$,

where $\tilde{r}(X) = \Delta\text{Encode}(r)$ and $\langle\langle r \rangle\rangle^{t,q,S_2} = ([r]^{t,q,S_2}, [\gamma_r]^{t,q,S_2})$ with $\gamma_r = \alpha r$.

Output: $\langle\langle x \rangle\rangle^{t,q,S_2} = ([x]^{t,q,S_2}, [\gamma_x]^{t,q,S_2})$ with $\gamma_x = \alpha x$.

Execution:

- Each $P_i^{(1)} \in \mathcal{S}_1$ computes $[c(X)]_i = b(X) - a(X)[s(X)]_i - [\tilde{r}(X)]_i + [e_{\text{flood}}(X)]_i$ and submits the share with its H2S transcript authentication.
- After validating a qualified set of submitted shares, P_0 reconstructs $c(X)$, extracts the target coefficient or slot $\bar{c} \in [0, Q)$, and broadcasts $\hat{\delta} = \lfloor \bar{c}/\Delta \rfloor \bmod q$ together with the reconstruction evidence. The committees abort if validation fails.
- Each $P_i^{(2)} \in \mathcal{S}_2$ computes

$$\begin{aligned} [x]_i &= [r]_i + \hat{\delta} \pmod{q}, \\ [\gamma_x]_i &= [\gamma_r]_i + \hat{\delta}[\alpha]_i \pmod{q}. \end{aligned}$$

The pair $([x]_i, [\gamma_x]_i)$ is retained as the authenticated output.

Fig. 13. Authenticated H2S conversion with handoff.

where $\gamma_r = \alpha r$. After the validated H2S reconstruction produces the public masked difference $\hat{\delta}$, the incoming committee uses linearity to obtain both the value and its MAC tag. This authenticates the SS output; binding that output to the input ciphertext additionally requires the share-validation and reconstruction checks of the underlying H2S transcript. To prepare the authenticated item, the offline phase first generates the base cross-domain pair from Eq. (23); $\mathcal{S}_2$ then uses the ordinary multiplication protocol from Eq. (18) to compute $[\gamma_r] = [\alpha r]$ without opening either r or α . The item is released only after this MAC relation has passed the preprocessing check.

D. Authenticated SS-to-FHE Conversion

Protocol 10 opens only an authenticated masked difference. The offline ciphertext $\llbracket r \rrbracket$ and authenticated sharing $\langle\langle r \rangle\rangle$ encode the same random field element. Before adding the opened difference to $\llbracket r \rrbracket$, the committee checks its MAC under the shared key α . Consequently, a dealer cannot replace the opened difference without causing the check to fail, except with the MAC-forgery probability of the underlying authenticated-sharing scheme. Its offline item is generated by first producing the base correlated pair $([r]^{t,q,S_1}, \llbracket r \rrbracket)$ and then attaching $[\gamma_r] = [\alpha r]$ by invoking the same ordinary

Protocol 10 $\llbracket x \rrbracket \leftarrow \Pi_{\text{auth-s2h}}(\langle\langle x \rangle\rangle^{t,q,S_1})$

Input: $\langle\langle x \rangle\rangle^{t,q,S_1} = ([x]^{t,q,S_1}, [\gamma_x]^{t,q,S_1})$ and $[\alpha]^{t,q,S_1}$, where $\gamma_x = \alpha x$.

Offline material: $(\langle\langle r \rangle\rangle^{t,q,S_1}, \llbracket r \rrbracket = (a(X), b(X)) \in \mathcal{R}_Q^2)$, where $\langle\langle r \rangle\rangle = ([r], [\gamma_r])$, $\gamma_r = \alpha r$, and both representations contain the same fresh $r \in \mathbb{F}_q$.

Output: Public ciphertext $\llbracket x \rrbracket \in \mathcal{R}_Q^2$.

Execution:

- Each $P_i^{(1)} \in \mathcal{S}_1$ computes $[\delta]_i = [x]_i - [r]_i$ and $[\gamma_\delta]_i = [\gamma_x]_i - [\gamma_r]_i$ in $\mathbb{F}_q$, and sends $[\delta]_i$ to P_0 .
- P_0 reconstructs and broadcasts $\bar{\delta} = (x - r) \bmod q \in [0, q)$. Each party computes $[v]_i = [\gamma_\delta]_i - \bar{\delta}[\alpha]_i$; the committee opens v and proceeds only if $v = 0$.
- After the MAC check succeeds, the parties set

$$\llbracket x \rrbracket = (a(X), b(X) + \Delta\text{Encode}(\bar{\delta}) \bmod Q).$$

Fig. 14. Authenticated S2H conversion.

multiplication protocol on $[r]$ and $[\alpha]$. Thus the ciphertext, value sharing, and MAC-tag sharing are bound to one fresh r and one conversion identifier.

E. Arithmetic-to-Boolean Share Conversion

Protocol 11 converts a Shamir sharing over $\mathbb{F}_q$ into Boolean sharings of the canonical binary representation of the same value. Its one-time BitExtract pair represents a uniformly random mask r both as an arithmetic sharing and as Boolean sharings of its bits. Opening $\delta = (x - r) \bmod q$ therefore reveals only a one-time-masked field element.

The parties add the public bits of δ to the secret bits of r with a ripple-carry circuit. Because this integer sum may equal either x or $x + q$, they conditionally subtract q and select the canonical result in $[0, q - 1]$. XOR gates and operations involving only public bits are local; each AND of two secret bits invokes the degree-reducing multiplication protocol over $\text{GF}(2^\kappa)$. The extension degree κ is chosen so that $2^\kappa - 1 \geq n$, providing at least n distinct nonzero Shamir evaluation points. Bits are embedded as the prime-subfield elements $\{0, 1\}$; because the extension field has characteristic two, addition of bits realizes XOR and multiplication realizes AND. The output bits consequently remain shared and can be consumed directly by subsequent Boolean MPC operators.

Prior work provides private ledger transitions, private contract execution, FHE-based encrypted state, threshold key operations, and dynamic MPC. The relevant integration problem is to combine public data identifiers with committee-held key shares, off-chain SS/FHE storage, contract-defined operators, and state transfer across changing executor groups.

a) *Proof- and hardware-based private execution:* Ben-Sasson *et al.* introduced Zerocash, which uses commitments, serial numbers, and zk-SNARKs to hide the origin, destina-

Protocol 11 $([x_0]^{t,2^\kappa,\mathcal{S}}, \dots, [x_{\ell-1}]^{t,2^\kappa,\mathcal{S}}) \leftarrow \Pi_{\text{S2B}}([x]^{t,q,\mathcal{S}})$

Input: A degree- t Shamir sharing $[x]^{t,q,\mathcal{S}}$ of $x \in \mathbb{F}_q$, held by the active committee $\mathcal{S}$, where $n > 2t$, $\ell = \lceil \log_2 q \rceil$, and x is represented by its canonical integer in $[0, q-1]$. Boolean wires use Shamir sharing over $\text{GF}(2^\kappa)$ with $2^\kappa - 1 \geq n$ and public evaluation points $\eta_1, \dots, \eta_n \in \text{GF}(2^\kappa)^*$.

Offline material: One BitExtract pair

$$([r]^{t,q,\mathcal{S}}, ([r_0]^{t,2^\kappa,\mathcal{S}}, \dots, [r_{\ell-1}]^{t,2^\kappa,\mathcal{S}})),$$

where $r \xleftarrow{\$} \mathbb{F}_q$ and $r = \sum_{k=0}^{\ell-1} 2^k r_k$ is its canonical binary decomposition. The pair is used once.

Output: Degree- t Shamir sharings over $\text{GF}(2^\kappa)$ of the canonical bits of x , held by $\mathcal{S}$, such that $x = \sum_{k=0}^{\ell-1} 2^k x_k$ as an integer in $[0, q-1]$.

Execution:

- Each $P_i \in \mathcal{S}$ computes $[\delta]_i^{t,q,\mathcal{S}} = [x]_i^{t,q,\mathcal{S}} - [r]_i^{t,q,\mathcal{S}}$ and sends its share to the designated dealer P_0 . The dealer reconstructs $\delta = (x - r) \bmod q$ and broadcasts δ to $\mathcal{S}$.
- The parties write the public value as $\delta = \sum_{k=0}^{\ell-1} 2^k \delta_k$ and evaluate a ripple-carry full-adder circuit on the public bits δ_k and the secret bits $[r_k]^{t,2^\kappa,\mathcal{S}}$. Starting from $[c_0] = 0$, for $k = 0, \dots, \ell-1$ they compute

$$[s_k] = \delta_k \oplus [r_k] \oplus [c_k],$$

$$[c_{k+1}] = (\delta_k \wedge [r_k]) \oplus (\delta_k \wedge [c_k]) \oplus ([r_k] \wedge [c_k]).$$

For every secret AND, the parties invoke $\Pi_{\text{DN-Mult}}$ over $\text{GF}(2^\kappa)$;

- The resulting $(\ell+1)$ -bit word represents the integer $s = \delta + r \in [0, 2q-2]$. Using the same full-adder gates, $\mathcal{S}$ computes the subtraction $s - q$ and its borrow bit, then multiplexes between s and $s - q$: it selects $s - q$ when $s \geq q$ and s otherwise. The low ℓ selected bits are output as $([x_0]^{t,2^\kappa,\mathcal{S}}, \dots, [x_{\ell-1}]^{t,2^\kappa,\mathcal{S}})$. Each secret AND in the subtractor and multiplexer likewise invokes $\Pi_{\text{DN-Mult}}$ and therefore interacts with P_0 .

Fig. 15. Secure S2B Protocol

tion, and amount of payments and preserve public verification [6]. Bowe *et al.* generalized proof-carrying ledger transitions through Zexe, whose decentralized private computation model hides offline computations and validates transactions succinctly [7]. Hawk separates a contract into public and private portions; a manager evaluates the private part and submits proofs and commitments that bind settlement to the contract semantics [8]. These systems establish strong public verifiability. Computation over private state held by several

users still requires a party or protocol to obtain a consistent witness for the relevant state transition.

Trusted hardware TEE-based systems evaluate private logic inside a hardware-isolated environment and post outputs, roots, or attestations to a ledger. Kaptchuk *et al.* show that a ledger can supply persistent state and publication evidence to an otherwise stateless trusted device [9]. This line of work clarifies rollback resistance and state freshness. Confidentiality continues to depend on the trusted-hardware boundary and its attestation and side-channel assumptions.

b) MPC- and FHE-based confidential contracts: MPC-based systems replace a single trusted manager with parties that jointly evaluate a private function over secret-shared inputs. zkHawk replaces Hawk's manager by MPC and avoids the cost of evaluating a complete zk-SNARK inside the MPC circuit [1]. V-zkHawk gives a UC-style variant using commitments, signatures, NIZKs, and broadcast assumptions [10]. Eagle combines outsourced MPC, Pedersen commitments, and zero-knowledge proofs to support long-running private contracts for clients that participate only during a short input phase [2]. These systems motivate distributed private execution. A changing executor set additionally requires explicit migration of the private state and its authentication material.

Homomorphic-encryption systems Zether uses additively homomorphic ElGamal encryption and zero-knowledge proofs to keep balances encrypted in an account-based smart-contract setting [3]. zkay introduces privacy types and transforms privacy-annotated contracts into public contracts enforced by encryption and NIZK proofs [4]. smartFHE uses FHE so that evaluators compute on encrypted inputs, and users prove input well-formedness [5]. Zama's fhEVM uses coprocessors for FHE evaluation, threshold MPC in a dedicated KMS for key generation and decryption, Gateway contracts for orchestration, and Solidity libraries for encrypted types [11]. Our system retains public identifiers for encrypted values. The active committee stores the FHE secret key in secret-shared form and executes contract-defined operators across the two representations.

c) Threshold decryption and dynamic participation:

Threshold FHE distributes key or decryption authority so that no single party holds the FHE secret key [16]. Noah's Ark studies threshold decryption for LWE-based FHE and uses noise flooding to hide the original ciphertext noise before a decryption-related value is opened [17]. Fluid secret sharing refreshes shares of an unchanged secret so that shares collected in different periods cannot be accumulated, provided that the corruption bound holds in every period and obsolete state is erased [18]. Fluid MPC formalizes MPC with dynamic participants and divides a long computation into committee epochs connected by state handoff [19]. PrivFlux combines public state references and off-chain ciphertext storage with committee-held key shares and fluid handoff. To reveal an FHE result, its authorized Pick first converts the ciphertext to an SS representation through H2S and then opens only that sharing to the named recipient; the FHE key shares remain committee-held.